👌 XAI Lecture 05

Rule Based Approaches

FAMAF - UNC

First Semester 2026

Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

https://interpretable-ml-class.github.io/

Paper 1 ---

The Annals of Applied Statistics
2015, Vol. 9, No. 3, 1350–1371
DOI: 10.1214/15-AOAS848

INTERPRETABLE CLASSIFIERS USING RULES AND BAYESIAN ANALYSIS: BUILDING A BETTER STROKE PREDICTION MODEL

BY BENJAMIN LETHAM[*,1], CYNTHIA RUDIN[*,1], TYLER H. MCCORMICK[†,2] AND DAVID MADIGAN[‡,3]

Massachusetts Institute of Technology[*], *University of Washington*[†] *and Columbia University*[‡]

We aim to produce predictive models that are not only accurate, but are also interpretable to human experts. Our models are decision lists, which consist of a series of *if*...*then*...statements (e.g., *if high blood pressure, then stroke*) that discretize a high-dimensional,

(Letham et al. 2015)

Contributions ---

- Introducing a generative model called **Bayesian Rule Lists (BRL)**
  - Goal is to output a decision list (`if then else-if`)
- Novel prior structure to **encourage sparsity**
  - "Sparse" means the model has few active elements
  - Instead of using all possible rules, BRL keeps only a few that are sufficiently expressive.
  - Truncated Poisson priors over list length (λ) and rule complexity (η) encourage *sparse*, *interpretable* decision lists.
- Predictive accuracy **on par** with top classic algorithms (RF, SVM, $CHADS_2$)

Decision List: Titanic Example ---

if male **and** adult **then** *survival probability* 21% (19%–23%)
else if 3rd class **then** *survival probability* 44% (38%–51%)
else if 1st class **then** *survival probability* 96% (92%–99%)
else *survival probability* 88% (82%–94%)

- This **is one of many** accurate and interpretable decision lists that can be learned from the data — BRL captures this uncertainty by maintaining a posterior distribution over all of them.
- **Each rule fires in order** — the first matching rule determines the prediction.
- A male adult in 1st class gets **21%**, not **96%**, because the first rule takes priority.
- The values in parentheses are **95% credible intervals** — the narrower, the more data behind that rule.

Implementing a Decision List in Python ---

```
>>> def predict_survival(male, adult, passenger_class):
...     if male and adult:
...         return 0.21, (0.19, 0.23)
...     elif passenger_class == 3:
...         return 0.44, (0.38, 0.51)
...     elif passenger_class == 1:
...         return 0.96, (0.92, 0.99)
...     return 0.88, (0.82 , 0.94)
...
>>> survival, confidence = predict_survival(True, False, 3)
>>> survival
0.44
>>> confidence
(0.38, 0.51)
```

Introduction: BRL ---

- New type of **balance** between accuracy, interpretability, and computation
- **What about using other similar models**?
  - Decision trees (CART)
  - They employ greedy construction methods
  - Not particularly computationally demanding but affects quality of solution – both accuracy and interpretability

Pre-mined Rules ---

- A major source of practical feasibility: **pre-mined rules**
  - Reduces model space
  - Complexity of problem depends on number of pre-mined rules
- As long as pre-mined set is expressive, **accurate decision list can be found** + smaller model space means **better generalization** (Vapnik, 1995)

Pre-mined Rules: Intuition ---

Itemsets
{1,2,3,4}
{1,2,4}
{1,2}
{2,3,4}
{2,3}
{3,4}
{2,4}

Item	Support
{1}	3
{2}	6
{3}	4
{4}	5

Item	Support
{1,2}	3
~~{1,3}~~	~~1~~
~~{1,4}~~	~~2~~
{2,3}	3
{2,4}	4
{3,4}	3

Item	Support
{2,3,4}	2

Minimum Support = 3

- So the final candidate antecedents passed to BRL would be: {2}, {3}, {4}, {2,3}, {2,4}, {3,4}.
- This is Apriori algorithm. FP-growth is a single pass algorithm (more efficient).

Preliminaries: Notation ---

- Training data $\{(x_i, y_i)\}_{i=1}^{n}$, $x_i \in \mathbb{R}^d$, $y_i \in \{1, \ldots, L\}$

$$\mathbf{x} = (x_1, \ldots, x_n) \quad \mathbf{y} = (y_1, \ldots, y_n)$$

Two labels:

1. stroke
2. no stroke

💻⚠️ Warning for students with a background in computer science

- **Intuition:** Bayesians use probability distributions like programmers use libraries (`numpy`, `sklearn`). Instead of building new theory from scratch, they pick standard distributions that fit their problem and update them with data.
- **Formally:** You encode prior beliefs using these distributions *before* seeing data, then combine them with observations via Bayes' theorem to get the posterior. The 'library' gives you the mathematical machinery, but you still need to choose the right prior and let the data update it.

Bayesian Decision Lists ---

$$
\begin{aligned}
&\textbf{if } a_1 \textbf{ then } y \sim \text{Multinomial}(\boldsymbol{\theta}_1),\ \boldsymbol{\theta}_1 \sim \text{Dirichlet}(\boldsymbol{\alpha} + \mathbf{N}_1) \\
&\textbf{else if } a_2 \textbf{ then } y \sim \text{Multinomial}(\boldsymbol{\theta}_2),\ \boldsymbol{\theta}_2 \sim \text{Dirichlet}(\boldsymbol{\alpha} + \mathbf{N}_2) \\
&\vdots \\
&\textbf{else if } a_m \textbf{ then } y \sim \text{Multinomial}(\boldsymbol{\theta}_m),\ \boldsymbol{\theta}_m \sim \text{Dirichlet}(\boldsymbol{\alpha} + \mathbf{N}_m) \\
&\textbf{else } y \sim \text{Multinomial}(\boldsymbol{\theta}_0),\ \boldsymbol{\theta}_0 \sim \text{Dirichlet}(\boldsymbol{\alpha} + \mathbf{N}_0).
\end{aligned}
$$

Where:

- a_j: Antecedent (the "if" condition) for rule j
- $y \sim \text{Multinomial}(\boldsymbol{\theta}_j)$: The predicted label follows a multinomial distribution with probabilities $\boldsymbol{\theta}_j$
- $\boldsymbol{\theta}_j \sim \text{Dirichlet}(\boldsymbol{\alpha} + \mathbf{N}_j)$: The parameters $\boldsymbol{\theta}_j$ themselves have a posterior distribution (Dirichlet) that combines the prior pseudocounts $\boldsymbol{\alpha}$ with the observed data counts $\mathbf{N}_j$
- $\boldsymbol{\alpha}$: Prior pseudocounts (hyperparameter). Setting $\boldsymbol{\alpha} = (1, 1, \ldots, 1)$ gives a uniform prior with no class preference
- $\mathbf{N}_j = (N_{j,1}, \ldots, N_{j,L})$: Counts of observations captured by rule j for each of the L classes

Preliminaries: Sampling from a multinomial ---

The code simulates throwing a 6-sided die 20 times, where each face has equal probability 1/6.

```
>>> import numpy as np
>>> np.random.multinomial(20, [1/6]*6, size=1)
array([[4, 1, 7, 5, 2, 1]])
```

- `[1/6] * 6 = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6]`
- returns how many times each face appeared: means face 1 appeared 4 times, face 2 once, face 3 seven times, etc.

Parameters are probability values

- θ_j, in BRL, is a vector of probabilities, one per class. For example, $\theta_j = [0.6, 0.3, 0.1]$ for a 3-class problem.
- When you sample a label y from Multinomial(θ_j), you're drawing one outcome according to those probabilities.

Preliminaries: Dirichlet Distribution ---

- **Dirichlet samples from this simplex:** Drawing from *Dirichlet*(α_1, α_2 gives probability vectors like (0.6, 0.4) or (0.3, 0.7)
- **Probability simplex:** The space of all valid probability vectors (sum to 1, non-negative)
  - Example: (0.6, 0.4) is valid; (0.6, 0.5) is not (sums to 1.1)
- **K-dimensional Dirichlet has k parameters** $(\alpha_1, \alpha_2, ..., \alpha_m)$
  - Each α_k controls expected probability mass for dimension k
  - Higher $\alpha_k \rightarrow$ more weight on that dimension
  - All $\alpha_k = 1 \rightarrow$ uniform distribution over the simplex

$$\Theta \sim \text{Dirichlet}(\alpha_1, \alpha_2, \ldots, \alpha_m)$$

$$P(\theta_1, \theta_2, \ldots, \theta_m) = \frac{\Gamma(\sum_k \alpha_k)}{\prod_k \Gamma(\alpha_k)} \prod_{k=1}^{m} \theta_k^{\alpha_k - 1}$$

Dirichlet as Conjugate Prior for Multinomial ---

- **Conjugate prior:** A prior is conjugate when the posterior has the same distributional form as the prior — no complex integrals needed.
- **Prior:**

$$(p_1, \ldots, p_k) \sim \text{Dirichlet}(\alpha_1, \ldots, \alpha_k)$$

- **Posterior:** Just add observed counts to prior pseudocounts:

$$(p_1, \ldots, p_k) \mid (x_1, \ldots, x_k) \sim \text{Dirichlet}(\alpha_1 + x_1, \ldots, \alpha_k + x_k)$$

🧉 Dirichlet as Conjugate Prior for Multinomial - Example ---

Prior

- **Prior** $\alpha = (1, 1, 1)$ (uniform, no class preference)
- $+$ observed counts $(5, 2, 3)$
- **Posterior** $=$ Dirichlet$(6, 3, 4)$

Why it matters for BRL:

Each rule's posterior is computed by simply adding observation counts to prior pseudocounts — clean and efficient.

Bayesian Association Rules ---

- A **Bayesian association rule** $a \to y$ predicts a label distribution instead of a single label:

$$a \to y \sim \text{Multinomial}(\boldsymbol{\theta})$$

- The parameters $\boldsymbol{\theta}$ themselves are uncertain — we put a Dirichlet prior on them:

$$\boldsymbol{\theta} \mid \boldsymbol{\alpha} \sim \text{Dirichlet}(\boldsymbol{\alpha})$$

- After observing data $(\mathbf{x}, \mathbf{y})$, the posterior is simply:

$$\boldsymbol{\theta} \mid \mathbf{x}, \mathbf{y}, \boldsymbol{\alpha} \sim \text{Dirichlet}(\boldsymbol{\alpha} + N)$$

- where $N = (N_{\cdot,1}, \ldots, N_{\cdot,L})$ are the observed counts per class for observations captured by this rule.

Bayesian Association Rules ---

$$a \to y \sim \text{Multinomial}(\boldsymbol{\theta}). \qquad \boldsymbol{\theta}|\boldsymbol{\alpha} \sim \text{Dirichlet}(\boldsymbol{\alpha}).$$

$$\boldsymbol{\theta}|\mathbf{x}, \mathbf{y}, \boldsymbol{\alpha} \sim \text{Dirichlet}(\boldsymbol{\alpha} + N)$$

$$N = (N_{\cdot,1}, \ldots, N_{\cdot,L})$$

Example

- Rule `"age > 60 AND diabetes"`
- Captures 10 patients: 7 with stroke, 3 without.
- With $\boldsymbol{\alpha} = (1, 1)$,
- posterior $=$ Dirichlet$(8, 4)$
- $\to$ estimated stroke probability $= 8/12 \approx 67\%$

Finally ---

At this point we know the ingredients:

- What a **Bayesian Rule List (BRL)** is: an ordered `if-then-else` decision list with probabilistic predictions
- How to mine a collection of candidate rules using **FP-Growth** (frequent itemset mining)
- How a **single rule** combines observations with a Dirichlet-Multinomial model to estimate class probabilities

Now it's time to put it all together and **learn the full BRL from data**.

Generative Model 1/3 ---

- Sample a decision list length $m \sim p(m|\lambda)$.
- Sample the default rule parameter $\theta_0 \sim \text{Dirichlet}(\alpha)$.
- For decision list rule $j = 1, \ldots, m$:
  Sample the cardinality of antecedent a_j in d as $c_j \sim p(c_j|c_{<j}, \mathcal{A}, \eta)$.
  Sample a_j of cardinality c_j from $p(a_j|a_{<j}, c_j, \mathcal{A})$.
  Sample rule consequent parameter $\theta_j \sim \text{Dirichlet}(\alpha)$.
- For observation $i = 1, \ldots, n$:
  Find the antecedent a_j in d that is the first that applies to x_i.
  If no antecedents in d apply, set $j = 0$.
  Sample $y_i \sim \text{Multinomial}(\theta_j)$.

Our goal instead of finding a single optimal list like a greedy algorithm, BRL assigns a **probability to every possible list** — giving us the full posterior distribution over decision lists.

$$p(d|\mathbf{x}, \mathbf{y}, \mathcal{A}, \boldsymbol{\alpha}, \lambda, \eta) \propto p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\alpha}) p(d|\mathcal{A}, \lambda, \eta).$$

continue...

Generative Model 2/3 ---

– Sample a decision list length $m \sim p(m|\lambda)$.
– Sample the default rule parameter $\theta_0 \sim \text{Dirichlet}(\alpha)$.
– For decision list rule $j = 1, \ldots, m$:
  Sample the cardinality of antecedent a_j in d as $c_j \sim p(c_j|c_{<j}, \mathcal{A}, \eta)$.
  Sample a_j of cardinality c_j from $p(a_j|a_{<j}, c_j, \mathcal{A})$.
  Sample rule consequent parameter $\theta_j \sim \text{Dirichlet}(\alpha)$.
– For observation $i = 1, \ldots, n$:
  Find the antecedent a_j in d that is the first that applies to x_i.
  If no antecedents in d apply, set $j = 0$.
  Sample $y_i \sim \text{Multinomial}(\theta_j)$.

$$p(d|\mathbf{x}, \mathbf{y}, \mathcal{A}, \boldsymbol{\alpha}, \lambda, \eta) \propto p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\alpha}) p(d|\mathcal{A}, \lambda, \eta).$$

The formula decomposes the posterior into two parts defined by the generative model:

- $p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\alpha})$: how well list d explains the data — comes from the **"For observation $i = 1, \ldots, n$"** step - **Likelihood**
- $p(d|\mathcal{A}, \lambda, \eta)$: how probable it was to construct that list — comes from the **"For decision list rule $j = 1, \ldots, m$"** step - **Prior**

In resume

- The generative model goes **list → data** ($p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\alpha})$), while the posterior goes **data → list** ($p(d|\mathbf{x}, \mathbf{y}, \mathcal{A}, \boldsymbol{\alpha}, \lambda, \eta)$)
- Bayes inverts the generative process.

Generative Model 3/3 ---

$$p(d|\mathbf{x}, \mathbf{y}, \mathcal{A}, \boldsymbol{\alpha}, \lambda, \eta) \propto p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\alpha}) p(d|\mathcal{A}, \lambda, \eta).$$

The posterior probability of list d, given the data $\mathbf{x}$, labels $\mathbf{y}$, candidate rules $\mathcal{A}$, and hyperparameters $\boldsymbol{\alpha}$(*pseudocounts*), λ, η, is proportional to the product of:

- $p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\alpha})$: the **likelihood** — how well list d explains the observed labels
- $p(d|\mathcal{A}, \lambda, \eta)$: the **prior** — how probable that list was before seeing any data
- λ controls the prior expected list length (number of `if then`),
- η controls the prior expected antecedent cardinality (number of conditions per rule).

***In short:** posterior $\propto$ likelihood $\times$ prior

Prior Probabilities ---

$$p(d|\mathcal{A}, \lambda, \eta) = p(m|\mathcal{A}, \lambda) \prod_{j=1}^{m} p(c_j|c_{<j}, \mathcal{A}, \eta) p(a_j|a_{<j}, c_j, \mathcal{A}).$$

List length prior (Truncated Poisson):

$$p(m|\mathcal{A}, \lambda) = \frac{(\lambda^m/m!)}{\sum_{j=0}^{|\mathcal{A}|}(\lambda^j/j!)}, \qquad m = 0, \ldots, |\mathcal{A}|$$

- Ensures that sampled values are within bounds! (in normal Poisson $m = 0, \ldots, \infty$)
- Also, ensures expected value is close to λ when there are a large number of pre-mined rules

Cardinality prior (Truncated Poisson)

$$p(c_j|c_{<j}, \mathcal{A}, \eta) = \frac{(\eta^{c_j}/c_j!)}{\sum_{k \in R_{j-1}(c_{<j}, \mathcal{A})}(\eta^k/k!)}, \qquad c_j \in R_{j-1}(c_{<j}, \mathcal{A}).$$

Antecedent selection

$p(a_j|a_{<j}, c_j, \mathcal{A})$ is sampled uniformly from available antecedents with appropriate cardinality. —

Prior Probabilities ---

$$p(d|\mathcal{A}, \lambda, \eta) = p(m|\mathcal{A}, \lambda) \prod_{j=1}^{m} p(c_j|c_{<j}, \mathcal{A}, \eta) p(a_j|a_{<j}, c_j, \mathcal{A}).$$

- **List length prior:**
  - Controls how many rules the list has (parameter λ)
  - Truncated at $|\mathcal{A}|$ instead of ∞ — list length can't exceed available rules
  - Expected value is close to λ when there are a large number of pre-mined rules
- **Cardinality prior:**
  - Controls how many conditions each rule has (parameter η)
  - Truncated to exclude cardinalities with no available rules at position j
- **Antecedent selection:**
  - Given a cardinality c_j, a rule is sampled **uniformly** from all available antecedents of that size

For each position in the list, the product picks a complexity (cardinality) and then a specific rule — repeated for all m rules.

Likelihood ---

- Likelihood is the product of multinomial probability mass functions for the observed label counts at each rule

$$p(\mathbf{y}|\mathbf{x}, d, \boldsymbol{\theta}) = \prod_{j:\sum_l N_{j,l}>0} \text{Multinomial}(\mathbf{N}_j|\theta_j), \qquad \theta_j \sim \text{Dirichlet}(\boldsymbol{\alpha})$$

- For each rule j that captures at least one observation, $\mathbf{N}_j$ counts how many times each class appeared — the multinomial measures how probable those counts are given θ_j
- **Marginalizing over θ_j:** θ_j is an intermediate parameter we don't need explicitly. Thanks to Dirichlet-Multinomial conjugacy, we can integrate it out analytically — the result depends only on observed counts $\mathbf{N}_j$ and prior $\boldsymbol{\alpha}$, with no need to estimate θ_j directly

Markov Chain Monte Carlo ---

- **Monte Carlo:** Generate random samples to approximate a distribution that is too complex to compute exactly
- **Markov:** Each proposed list d^* depends only on the current list d^t — not on the entire history of the chain
- **Chain:** Samples are sequential stepping stones — each one is used to propose the next
- After enough iterations, the chain **converges** to the true posterior distribution over decision lists

Markov Chain Monte Carlo: Proposal Moves ---

- At each iteration, propose a new list d^* from the current d^t by randomly choosing one of three moves:
  - **Move:** relocate an existing rule to a different position in the list
  - **Add:** insert a new rule from $\mathcal{A}$ not currently in d^t
  - **Remove:** delete a rule from d^t
- The proposed d^* is then accepted or rejected via the **Metropolis-Hastings** criterion

Metropolis-Hastings ---

- **Initialize:** start with a random decision list d^0 sampled from the prior
- **Propose:** sample a new list d^* from the proposal distribution $Q(d^*|d^t)$ using one of the three moves (add, remove, move)
- **Accept/Reject:** compute the acceptance probability:

$$A = \min\left(1, \frac{p(d^*|\mathbf{x}, \mathbf{y}, \mathcal{A}) \cdot Q(d^t|d^*)}{p(d^t|\mathbf{x}, \mathbf{y}, \mathcal{A}) \cdot Q(d^*|d^t)}\right), \quad A \in (0, 1]$$

- **Sample:**
  - **draw** $\{u \sim \text{Uniform}(0, 1)\}$
  - **if** $u \leq A$
    - ⋆ **if-True: set** $d^{t+1} = d^*$
    - ⋆ **otherwise: keep** $d^{t+1} = d^t$
- **Repeat** until convergence

Metropolis-Hastings: Acceptance Probability ---

$$A = \min\left(1, \frac{p(d^*|\mathbf{x}, \mathbf{y}, \mathcal{A}) \cdot Q(d^t|d^*)}{p(d^t|\mathbf{x}, \mathbf{y}, \mathcal{A}) \cdot Q(d^*|d^t)}\right), \quad A \in (0, 1]$$

The formula compares how good the proposed list d^* is vs. the current list d^t. The ratio has two parts:

- **Posterior ratio** $p(d^*|\ldots)/p(d^t|\ldots)$: how much better d^* is in terms of posterior — if d^* has higher posterior, this ratio is > 1
- **Proposal ratio** $Q(d^t|d^*)/Q(d^*|d^t)$: corrects for the fact that some moves are easier to propose than others

The $\min(1, \ldots)$:

- If the ratio ≥ 1: always accept ($A = 1$) — d^* is better
- If the ratio < 1: accept with probability equal to the ratio — d^* is worse but not discarded completely

Accepting worse solutions with some probability prevents getting trapped in local optima.

Proposal Probabilities ---

- Move chosen uniformly
- Which antecedents and their new position is also chosen uniformly

$$Q(d^*|d^t, \mathcal{A}) = \begin{cases} \frac{1}{(|d^t|)(|d^t|-1)}, & \text{if move proposal,} \\ \frac{1}{(|\mathcal{A}|-|d^t|)(|d^t|+1)}, & \text{if add proposal,} \\ \frac{1}{|d^t|}, & \text{if remove proposal.} \end{cases}$$

Naive Python BRL creation ---

```
A = fp_growth(data, min_support=0.1, max_cardinality=2) # Step 1: Mine candidate rules
d = sample_from_prior(A, lambda_=3, eta=1) # Step 2: Initialize a random decision list from the prior

for iteration in range(n_iterations): # Step 3: MCMC - sample from posterior

    # Propose a new list by randomly choosing one of three moves:
    move = random.choice(["move", "add", "remove"])

    if move == "move": d_proposed = move_rule(d) # move a rule to a different position
    elif move == "add": d_proposed = add_rule(d, A) # add a new rule from A
    else: d_proposed = remove_rule(d) # remove a rule from d

    # Compute acceptance probability (posterior ratio)
    likelihood_current = compute_likelihood(d, X, y, alpha)  # uses N_j and alpha directly
    likelihood_proposed = compute_likelihood(d_proposed, X, y, alpha)
    prior_current = compute_prior(d, A, lambda_, eta)
    prior_proposed = compute_prior(d_proposed, A, lambda_, eta)

    acceptance = min(1, (likelihood_proposed * prior_proposed) /
                        (likelihood_current  * prior_current))

    d = d_proposed if random.uniform(0, 1) < acceptance else d # Accept or reject
    posterior_samples.append(d)

# Step 4: Extract point estimate
d_hat = brl_point(posterior_samples)
```

Estimating label of a new observation ---

$$p(\tilde{y} = l \mid \tilde{x}, d, \mathbf{x}, \mathbf{y}, \boldsymbol{\alpha}) = \frac{\alpha_l + N_{j(d,\tilde{x}),l}}{\sum_{k=1}^{L}(\alpha_k + N_{j(d,\tilde{x}),k})}$$

- $j(d, \tilde{x})$: index of the **first rule in d that matches** the new observation $\tilde{x}$
- **Numerator:** observed counts for class l in that rule + prior pseudocount α_l
- **Denominator:** total counts across all classes + all priors — ensures the result is a valid probability
- This is the Dirichlet-Multinomial posterior evaluated at the matched rule

🧉 Prediction: Estimating the Label of a New Observation ---

Once MCMC has converged and we have a point estimate $\hat{d}$:

1. Find the **first rule** in $\hat{d}$ that matches the new observation $\tilde{x}$
2. Retrieve the **precomputed counts** N_j for that rule
3. Compute the **posterior probability** for each class:

```python
def predict_label(N, alpha, L=2):
    total = sum(alpha[k] + N[k] for k in range(L))
    probs = [(alpha[l] + N[l]) / total for l in range(L)]
    return probs

# Rule j captured 7 strokes and 3 no-strokes
N     = [7, 3]
alpha = [1, 1]   # uniform prior

probs = predict_label(N, alpha)
print(f"Stroke probability:    {probs[0]:.2%}")   # (1+7)/(12) = 66.67%
print(f"No-stroke probability: {probs[1]:.2%}")   # (1+3)/(12) = 33.33%
```

Experiment: Tic-Tac-Toe ---

	BRL	C5.0	CART	ℓ_1-LR	SVM	RF	BCART
Mean accuracy	1.00	0.94	0.90	0.98	0.99	0.99	0.71
Standard deviation	0.00	0.01	0.04	0.01	0.01	0.01	0.04

5 fold cross validation; accuracy computed across 5 folds

Stroke Prediction - $CHADS_2$: The Baseline ---

Factor	Points
C — Congestive heart failure	1
H — Hypertension	1
A — Age $\geq$ 75	1
D — Diabetes mellitus	1
$\mathbf{S_2}$ — Prior stroke/TIA	2

- Higher score $\rightarrow$ higher stroke risk
- Designed **manually** by clinical experts
- Calibrated on only **1,733 patients** and **5 variables**

Stroke Prediction ---

- N = 12,586, 14% had stroke
- 6000 times larger than data for $CHADS_2$ score
- 5 fold evaluation
- Pre-mining: support 10% and max cardinality 2

Pre-mining Example

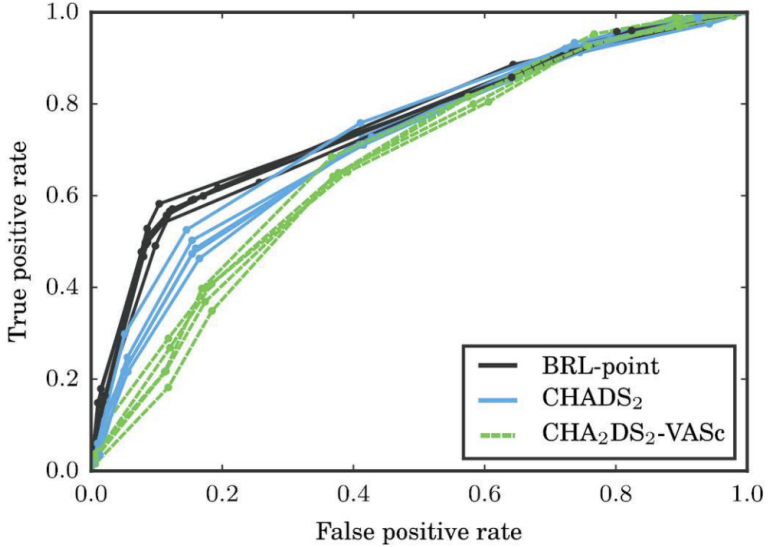
From 4,148 variables, FP-Growth with support $\geq 10\%$ and max cardinality 2 reduces the candidate pool to $\approx 2,200$ antecedents:

- `diabetes AND age > 60` ✓ (cardinality 2, appears in $\geq 10\%$ of patients)
- `hypertension` ✓ (cardinality 1, frequent)
- `diabetes AND age > 60 AND hypertension` × (cardinality 3, excluded)
- `rare_condition` × (support $< 10\%$, excluded)

Stroke Prediction ---

if hemiplegia **and** age $>$ 60 **then** *stroke risk* 58.9% (53.8%–63.8%)
else if cerebrovascular disorder **then** *stroke risk* 47.8% (44.8%–50.7%)
else if transient ischaemic attack **then** *stroke risk* 23.8% (19.5%–28.4%)
else if occlusion and stenosis of carotid artery without infarction
then *stroke risk* 15.8% (12.2%–19.6%)
else if altered state of consciousness **and** age $>$ 60
then *stroke risk* 16.0% (12.2%–20.2%)
else if age $\leq$ 70 **then** *stroke risk* 4.6% (3.9%–5.4%)
else *stroke risk* 8.7% (7.9%–9.6%)

Stroke Prediction - ROC ---



Stroke Prediction - AUC ---

	AUC	Training time (mins)
BRL-point	0.756 (0.007)	21.48 (6.78)
$CHADS_2$	0.721 (0.014)	no training
CHA_2DS_2-VASc	0.677 (0.007)	no training
CART	0.704 (0.010)	12.62 (0.09)
C5.0	0.704 (0.011)	2.56 (0.27)
ℓ_1 logistic regression	0.767 (0.010)	0.05 (0.00)
SVM	0.753 (0.014)	302.89 (8.28)
Random forests	0.774 (0.013)	698.56 (59.66)

Note

- AUC may be misleading here — the dataset is heavily imbalanced (14% stroke, 86% no stroke). Metrics such as AUPRC or F1-score would better capture model performance on the minority class.
- Additionally, AUC treats false positives and false negatives equally, which is inappropriate in a medical setting where missing a stroke is far costlier than a false alarm.

Interpretable Decision Sets: A Joint Framework for Description and Prediction

Himabindu Lakkaraju
Stanford University
himalv@cs.stanford.edu

Stephen H. Bach
Stanford University
bach@cs.stanford.edu

Jure Leskovec
Stanford University
jure@cs.stanford.edu

ABSTRACT

One of the most important obstacles to deploying predictive models is the fact that humans do not understand and trust them. Knowing which variables are important in a model's prediction and how they are combined can be very powerful in helping people understand and trust automatic decision making systems.

Here we propose interpretable decision sets, a framework for building predictive models that are highly accurate, yet also highly

predicted for different data points [44]. When there are multiple classes to predict, it is also important to characterize all the classes, not just the common ones.

Interpretable models are needed in many domains because they bridge the gap between domain experts and data scientists. When domain experts need to make important decisions, learning from data can improve their results, but this requires the human to understand and trust the model. From medical diagnosis to decision

(Lakkaraju, Bach, and Leskovec 2016)

Contributions ---

- A framework called **Interpretable Decision Sets** (IDS) for classification
- **Novel objective function** + proof of **submodularity**
- Optimization procedure with optimality guarantees
- **Detailed metrics for evaluating interpretability** + user studies

Motivation ---

- Traditional classification models optimize for predictive accuracy
- Very little understanding of the model itself and its predictions
- Model being “readable” is not enough
- **Humans should be able to reason about predictions and readily explain the functionality of the model**

Decision Sets ---

If Respiratory-Illness=Yes **and** Smoker=Yes **and** Age$\geq$ 50 **then** Lung Cancer

If Risk-LungCancer=Yes **and** Blood-Pressure$\geq$ 0.3 **then** Lung Cancer

If Risk-Depression=Yes **and** Past-Depression=Yes **then** Depression

If BMI$\geq$ 0.3 **and** Insurance=None **and** Blood-Pressure$\geq$ 0.2 **then** Depression

If Smoker=Yes **and** BMI$\geq$ 0.2 **and** Age$\geq$ 60 **then** Diabetes

If Risk-Diabetes=Yes **and** BMI$\geq$ 0.4 **and** Prob-Infections$\geq$ 0.2 **then** Diabetes

If Doctor-Visits $\geq$ 0.4 **and** Childhood-Obesity=Yes **then** Diabetes

Decision Lists vs Decision Sets ---

Decision Lists

- Ordered sequence of `if-then-else` rules
- Order matters: first matching rule applies
- Like a chain of `if/else-if` statements

Decision Sets

- Unordered collection of `if-then` rules
- Each rule independently assigns a class
- Rules can overlap (multiple rules may fire)

Criteria for Interpretability ---

- **Parsimony**: Fewer rules with fewer conditions
  - Cognitive limits of human understanding
- **Distinctness**: Minimal overlap of rules w.r.t the data points they cover
  - No redundant and contradicting explanations of data points
- **Class Coverage**: Explain all the classes in the data
  - Rules explaining minority classes are important

Interpretable Decision Sets: Problem Setup ---

Input:

- Set of data points: $\mathcal{D} = \{(\mathbf{x}_1, y_1), \cdots, (\mathbf{x}_N, y_N)\}$
- Set of class labels: $\mathcal{C}$
- Set of frequent itemsets obtained via Apriori: $\mathcal{S} = \{s_1, s_2, s_3, \ldots, s_M\}$
  - Ex: $s_i =$ `Gender = Female AND Age > 35 AND Smoker = True`

Output: An interpretable decision set $\mathcal{R} \subseteq \mathcal{S} \times \mathcal{C}$

- Each rule in $\mathcal{R}$ is a pair (itemset, class) — you pick an itemset from $\mathcal{S}$ and assign it a class label from $\mathcal{C}$
- $\mathcal{R}$ is a subset of all possible itemset-class combinations $\mathcal{S} \times \mathcal{C}$

We need to optimize for the following criteria for...

Acurate Predictions

- **Recall:** Fraction of data points correctly covered by some rule
- **Precision:** Fraction of covered points that belong to the correct class

Interpretability

- **Distinctness:** Rules cover non-overlapping regions of the feature space
- **Parsimony:** Decision set has few short rules
- **Class Coverage:** At least one rule exists for each class

Objective Function - Parsimony 1/2 ---

Number of rules

- Fewer rules: $f_1(\mathcal{R}) = |\mathcal{S}| - \text{size}(\mathcal{R})$

Number of conditions in the rule

- Fewer predicates:

$$f_2(\mathcal{R}) = L_{\max} \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} \text{length}(r)$$

Maximum no. of conditions in any given Input pattern

Total number of input patterns

Objective Function - Parsimony 2/2 ---

Fewer rules (f_1)

The fewer rules in the decision set, the easier it is for a user to understand all the conditions for each class. Since $|\mathcal{S}|$ is constant, **maximizing** $f_1(\mathcal{R}) = |\mathcal{S}| - \textbf{size}(\mathcal{R})$ is equivalent to minimizing the number of rules.

- Prefer: 12 rules over 30 rules

Fewer predicates (f_2)

The fewer conditions inside each rule, the easier each rule is to parse. Since L_{max} and $|\mathcal{S}|$ are constant, **maximizing** $f_2(\mathcal{R}) = L_{\text{max}} \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} \textbf{length}(r)$ is equivalent to minimizing the total number of predicates across all rules.

- Prefer: `Age > 60` (length 1) over `Age > 60 AND Diabetes AND Hypertension AND Smoker` (length 4)

Objective Function - Distinctness 1/2 ---

- Intra-class overlap:

$$f_3(\mathcal{R}) = N \cdot |S|^2 - \sum_{\substack{r_i, r_j \in \mathcal{R} \\ i \leq j \\ c_i = c_j}} \text{overlap}(r_i, r_j)$$

Total number of data points

Number of points that satisfy both the rules

- Inter-class overlap:

$$f_4(\mathcal{R}) = N \cdot |S|^2 - \sum_{\substack{r_i, r_j \in \mathcal{R} \\ i \leq j \\ c_i \neq c_j}} \text{overlap}(r_i, r_j)$$

Low intra-class overlap (f_3)

Two rules predicting the same class should not cover the same data points. Since N and $|\mathcal{S}|^2$ are constant, maximizing $f_3(\mathcal{R}) = N \cdot |\mathcal{S}|^2 - \sum_{r_i, r_j \in \mathcal{R}, c_i = c_j} \text{overlap}(r_i, r_j)$ is equivalent to minimizing overlap among rules of the same class.

- Prefer: two diabetes rules covering different patients over two diabetes rules covering the same patients

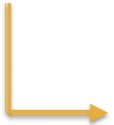
Low inter-class overlap (f_4) ⚠

Two rules predicting different classes should not cover the same data points — otherwise the tie-breaking function must decide, reducing interpretability. Since N and $|\mathcal{S}|^2$ are constant, maximizing $f_4(\mathcal{R}) = N \cdot |\mathcal{S}|^2 - \sum_{r_i, r_j \in \mathcal{R}, c_i \neq c_j} \text{overlap}(r_i, r_j)$ is equivalent to minimizing overlap among rules of different classes.

- Prefer: a diabetes rule and a depression rule covering different patients over both covering the same patients

Objective Function - Class Coverage ---

$$f_5(\mathcal{R}) = \sum_{c' \in \mathcal{C}} 1\left(\exists r = (s, c) \in \mathcal{R} \text{ such that } c = c'\right)$$



Check if there exists some rule corresponding to a given class c

At least one rule must exist for every class in the data. This is especially important for rare but critical classes that might otherwise be ignored in favor of more common ones.

- f_5 counts how many classes have at least one rule predicting them — maximized when every class is covered by some rule in $\mathcal{R}$
- Maximized when every class has at least one rule predicting it
- Prefer: a decision set that covers all 6 diseases over one that ignores rare blood cancers like Leukemia

Objective Function - **Precision** ---

- Minimize "incorrect" covers:

$$f_6(\mathcal{R}) = N \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} |\text{incorrect-cover}(r)|$$

Given a rule $r = (s, c)$, the no. of data points which satisfy s but do not belong to class c.

Rules should cover as few incorrectly labeled data points as possible. Since N and $|\mathcal{S}|$ are constant, maximizing $f_6(\mathcal{R}) = N \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} |\text{incorrect-cover}(r)|$ is equivalent to minimizing the number of data points incorrectly covered by each rule.

- Prefer: a diabetes rule that covers 90 diabetic and 2 non-diabetic patients over one that covers 90 diabetic and 30 non-diabetic patients

Objective Function - **Recall** ---

- Encourage at least one "correct" cover per data point:

$$f_7(\mathcal{R}) = \sum_{(\mathbf{x},y)\in\mathcal{D}} 1\left(\left|\{r|(\mathbf{x},y)\in \text{correct-cover}(r)\}\right| \geq 1\right)$$

Given a rule $r = (s,c)$, the no. of data points which satisfy s and belong to class c.

The decision set should correctly cover as many data points as possible with at least one rule.

- Maximized when every data point is correctly covered by at least one rule
- Prefer: a decision set that correctly classifies 95% of patients over one that leaves 30% uncovered

Objective Function ---

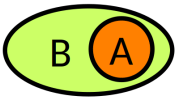
- Complete objective is

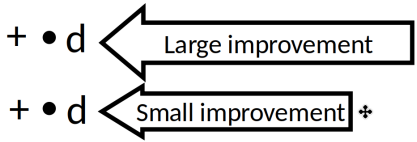
$$\underset{\mathcal{R}\subseteq\mathcal{S}\times\mathcal{C}}{\textbf{argmax}}\ \Sigma_{i=1}^{7}\ \lambda_i f_i(\mathcal{R})$$

- The intra-class and inter-class overlap terms are **non-monotone**
  - Adding more rules to $\mathcal{R}$ does not always improve f_3 and f_4. In fact, adding a rule can increase overlap and worsen the objective.
- The parsimony, overlap, and precision terms are **non-normal**
  - A function is "normal" if $f(\emptyset) = 0$. Here $f_3(\emptyset) = N \cdot |\mathcal{S}|^2 \neq 0$ — the empty set already has a positive value because the penalty terms vanish. The same applies to parsimony and precision.
- All the component terms are **submodular**

Diminishing returns characterization

$$\underbrace{F(A \cup d) - F(A)}_{\text{Gain of adding } \boldsymbol{d} \text{ to a small set}} \geq \underbrace{F(B \cup d) - F(B)}_{\text{Gain of adding } \boldsymbol{d} \text{ to a large set}}$$

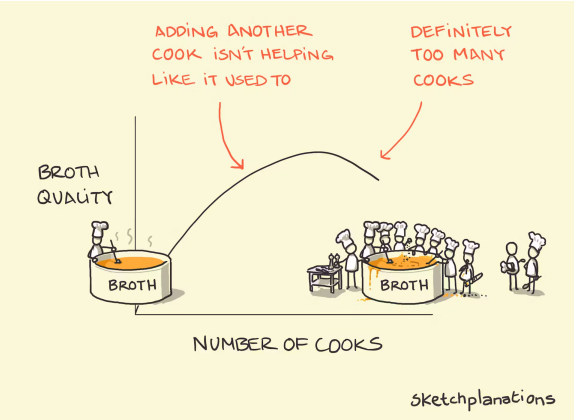




Key Property

A non-negative linear combination of submodular functions is submodular

Sublodularity ---



submodularity **is** diminishing returns, **but for set functions** instead of continuous ones.

Objective Function ---

- Complete objective is

$$\underset{\mathcal{R} \subseteq \mathcal{S} \times \mathcal{C}}{\textbf{argmax}} \; \Sigma_{i=1}^{7} \lambda_i f_i(\mathcal{R})$$

- **Non-negative**
- **Non-normal**
- **Non-monotone**
- **Submodular**
  - Adding a rule r to a small $\mathcal{R}$ yields a large gain (uncovered space, little overlap, few classes represented), while adding the same rule to a large $\mathcal{R}$ yields a smaller gain.

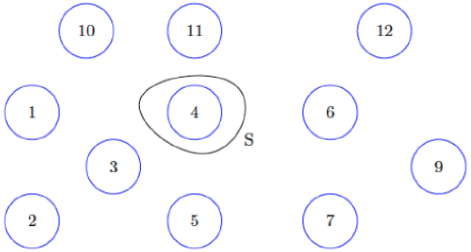
Optimizing the Objective ---

- Maximizing a non-monotone submodular function is **NP-hard**
  - The space of possible decision sets is exponential in $|\mathcal{S}|$ — exhaustive search is intractable
- **Smooth Local Search (SLS)** algorithm provides a $2/5$ approximation [Feige, Mirrokni, Vondrak FOCS 07; SIAM Comp. J. 11]
  - Stochastically adds and removes rules based on their estimated marginal contribution
  - Guarantees a solution worth **at least** $2/5$ **of the optimal** — e.g., if the best possible score is 100, SLS finds a solution with score ≥ 40
  - Unlike MCMC-based methods (e.g., BRL), SLS provides a **formal theoretical guarantee** on solution quality

Submodular Maximization: Local Search 1/7 ---

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \backslash \{v\}$.



$$f(S) = 10 \quad S = \{4\}$$

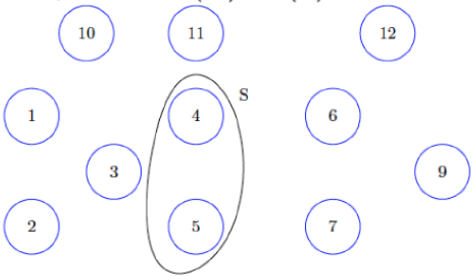
Each node here corresponds to a candidate
rule = (pattern, class) tuple

S and S' correspond to the intermediate solution sets

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \backslash \{v\}$.

Improving local operation if $f(S') > f(S)$.



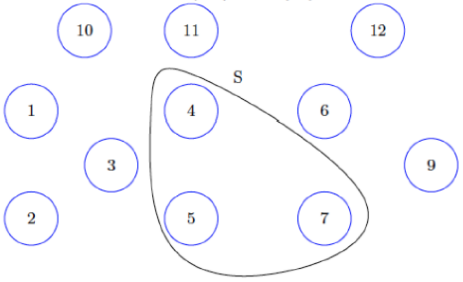
$$S = \{4, 5\}$$
$$f(S) = 17$$

S and S' correspond to the intermediate solution sets

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \setminus \{v\}$.

Improving local operation if $f(S') > f(S)$.



$S = \{4, 5, 7\}$

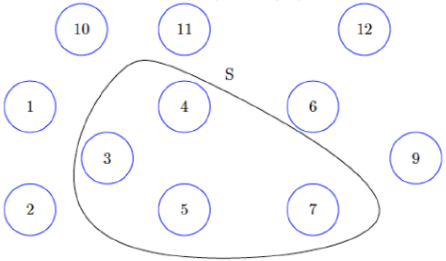
$f(S) = 23$

S and S' correspond to the intermediate solution sets

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \backslash \{v\}$.

Improving local operation if $f(S') > f(S)$.



$S = \{4, 5, 7, 3\}$

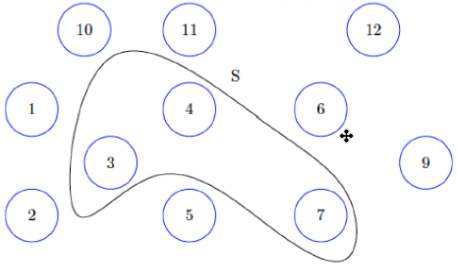
$f(S) = 27$

S and S' correspond to the intermediate solution sets

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \backslash \{v\}$.

Improving local operation if $f(S') > f(S)$.



$S = \{4, 7, 3\}$

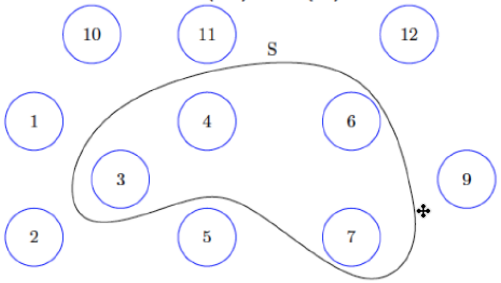
$f(S) = 30$

S and S' correspond to the intermediate solution sets

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \backslash \{v\}$.

Improving local operation if $f(S') > f(S)$.



$S = \{4, 7, 3, 6\}$

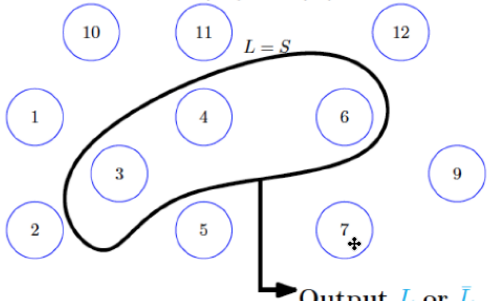
$f(S) = 33$

S and S' correspond to the intermediate solution sets

Local Operation:

- Add v: $S' = S \cup \{v\}$.
- Remove v: $S' = S \backslash \{v\}$.

Improving local operation if $f(S') > f(S)$.



S and S' correspond to the intermediate solution sets

- **Standard submodular maximization** gives a $1/3$ approximation — the solution is guaranteed to be at least $1/3$ of the optimal
- **IDS uses Smooth Local Search (SLS)**, a stronger variant that achieves a $2/5$ approximation
  - Stochastically adds and removes rules based on estimated marginal contributions
  - Tighter guarantee: solution is at least $2/5$ of optimal (e.g., score ≥ 40 if optimal $= 100$)

Smooth Local Search ---

Algorithm 1 Smooth Local Search (SLS)

1: **Input:** Objective f, domain $X = \mathcal{S} \times \mathcal{C}$, parameters δ and δ'
2:
3: $A = \emptyset$
4: $\texttt{OPT} = f(\Phi_X(X, 0))$
5: **for** each element $x \in X$ **do**
6: Estimate $\mathbb{E}[f(\Phi_X(A, \delta) \cup x)] - \mathbb{E}[f(\Phi_X(A, \delta) \setminus x)]$ within an error of $\frac{1}{|X|^2}\texttt{OPT}$
7: Call this estimate $\tilde{\omega}_{A,\delta}(x)$
8: **end for**
9: **for** each element $x \in X \setminus A$ such that $\tilde{\omega}_{A,\delta}(x) > \frac{2}{|X|^2}\texttt{OPT}$ **do**
10: $A = A \cup x$
11: Goto Line 5
12: **end for**
13: **for** each element $x \in A$ such that $\tilde{\omega}_{A,\delta}(x) < \frac{-2}{|X|^2}\texttt{OPT}$ **do**
14: $A = A \setminus x$
15: Goto Line 5
16: **end for**
17: **return** $\Phi_X(A, \delta')$

Initialization

Marginal gain

If marginal gain > threshold, Add element

If marginal gain < threshold, Add element

Smooth Local Search (SLS): Intuition ---

Goal: Find a good decision set without exploring all $2^{|\mathcal{S}|}$ possibilities.

Algorithm (simplified):

1. **Initialize:** start with an empty set $A = \emptyset$
2. **Estimate:** for each candidate rule, estimate how much it improves the objective if added or removed
3. **Add:** if a rule improves the objective enough $\rightarrow$ add it to A
4. **Remove:** if a rule hurts the objective enough $\rightarrow$ remove it from A
5. **Repeat** until no rule can be added or removed
6. **Return** a random subset of A (the "smoothing" step)

Key insight: The smoothing step prevents getting trapped in local optima — similar in spirit to Metropolis-Hastings in BRL, but with a **formal 2/5 optimality guarantee**.

Submodular Optimization vs. SLS ---

	Standard Submodular	SLS
Function type	Monotone	Non-monotone, non-normal
Approximation	1/2 greedy	2/5
Adding rules always helps?	Yes	No (overlap can worsen objective)
Used in IDS?	No	Yes

Standard Submodular is the general idea, **SLS** is one implementation.

Evaluation: Datasets ---

Dataset	# Datapoints	Features	Classes
Bail Outcomes	86K	Gender, age, current offense details, past criminal record	No Risk, Failure to Appear, New Criminal Activity
Student Performance	21K	Gender, age, grades, absence rates & tardiness, suspension/withdrawal history (grades 6-8)	Graduated on Time, Delayed Graduation, Dropped Out
Medical Diagnosis	150K	Current ailments, age, BMI, gender, smoking habits, medical history, family history	Asthma, Diabetes, Depression, Lung Cancer, Rare Blood Cancer

Evaluating Predictive Performance - **AUC** ---

Method	Bail	Student	Medical
IDS	**69.78**	**75.12**	**61.19**
Bayesian Decision Lists (Letham et al.)	67.18	72.54	59.18
Classification Based on Association (Liu et al.)	70.68	76.02	63.03
CN2	71.02	76.36	64.78
Decision Trees	70.08	75.31	63.28
Gradient Boosted Trees	71.23	77.18	64.21
Random Forests	70.87	77.12	63.92

Note

The paper does not report class distributions for any of the datasets. Given the high-stakes nature of the problems (justice, education, medicine), **class imbalance is likely** — results should be interpreted with **caution**, as AUC may be misleading in such settings.

Evaluating Goodness of Rules on Medical Diagnosis Data ---

Method	Frac. Overlap	Frac. Uncovered	Avg. Rule Length	Num. Rules	Frac. Classes
IDS	**0.09**	**0.13**	**3.17**	**12**	**1.00**
BDL	0.00	0.18	8.46	11	0.67
CBA	0.00	0.14	8.60	32	1.00
CN2	0.12	0.14	9.78	38	1.00

IDS achieves the best balance across all interpretability metrics: rules are **3x shorter** than baselines, cover **all 6 classes** (BDL misses 2 rare diseases), and uses far **fewer rules** than CBA and CN2. The small overlap (0.09) is a deliberate trade-off — unlike BDL and CBA whose zero overlap comes from their rigid if-then-else structure, not from interpretability design.

Ablation Study: Results on Medical Diagnosis Data ---

Method	AUC	Frac. Overlap	Frac. Uncov.	Rule Length	Num. Rules	Frac. Classes
Full IDS	61.19	0.09	0.13	3.17	12	1.00
No Precision	51.26	0.09	0.19	3.19	12	1.00
No Recall	53.38	0.10	0.14	3.18	11	1.00
No Overlap	61.02	0.16	0.14	3.54	11	1.00
No Conciseness	63.64	0.04	0.13	6.88	14	1.00
No Class	59.28	0.01	0.15	3.09	10	0.83

Each row removes one component of the objective. **Precision and recall** drive accuracy — removing either causes a large AUC drop. **Overlap** constraints keep rules distinct — without them overlap doubles. **Conciseness** keeps rules short — without it rule length more than doubles. **Class coverage** ensures rare diseases are represented — without it one class is missed entirely.

Evaluating Interpretability: User Study ---

- **47 Stanford data mining students** randomly assigned to either IDS or BDL — never both
- **12 questions per user:** 10 objective (true/false) + 2 descriptive (written paragraph)
- **Objective questions:** given partial patient attributes, can you conclude the patient has a specific disease?
- **Descriptive questions:** write a paragraph describing all characteristics of patients with a specific disease

  The key challenge in descriptive questions was correctly parsing conjunctions, disjunctions, and if-then/if-then-else structure. BDL users most commonly failed by **not accounting for negations induced by preceding rules** *— a direct consequence of the if-then-else structure.*

User Study Results ---

Task	Metric	IDS	BDL
Descriptive	Human Accuracy	0.81	0.17
	Avg. Time (secs.)	113.4	396.86
	Avg. # of Words	31.11	120.57
Objective	Human Accuracy	0.97	0.82
	Avg. Time (secs.)	28.18	36.34

- **Objective questions:** IDS users were 17% more accurate and 22% faster
- **Descriptive questions:** IDS users used 74% fewer words and were 71% faster — and **4.7x more accurate** (0.81 vs 0.17)
  The dramatic gap in descriptive accuracy confirms that decision lists are fundamentally harder to interpret: understanding a rule requires reasoning about **all preceding rules** *simultaneously, which humans consistently fail to do correctly.*

BRL Vs. IDS - When to Use Each Model? ---

	BRL	IDS
Structure	Ordered if-then-else list	Unordered if-then rules
Prediction	Probabilistic (credible intervals)	Deterministic (precision-based)
Best for	Binary classification, medical risk scoring	Multi-class problems with rare classes
Uncertainty	Built-in via Dirichlet-Multinomial	Not provided
Optimization	MCMC sampling	Submodular optimization (2/5 guarantee)
Rule overlap	Zero by construction	Minimized but allowed
Use when	You need calibrated probabilities and credible intervals	You need equally good rules for all classes

General Discussion ---

Comparing Both Papers:

- **BRL (Letham et al.):** Bayesian decision lists, probabilistic predictions, medical domain, MCMC sampling
- **IDS (Lakkaraju et al.):** Decision sets, joint optimization of accuracy and interpretability, formal approximation guarantees, user study validation

Key Lessons:

- Interpretability is **multifaceted** — sparsity, overlap, coverage, and rule length all matter
- **Ordered vs. unordered rules** is not just a structural choice — it fundamentally affects how humans understand models
- Both papers show that interpretability and accuracy are **not mutually exclusive**
- Evaluating interpretability requires **both quantitative metrics and user studies**
- The right model depends on the task: BRL excels at **uncertainty quantification**, IDS at **multi-class clarity**

Thank You!

References --- I

Lakkaraju, Himabindu, Stephen H. Bach, and Jure Leskovec. 2016. "Interpretable Decision Sets: A Joint Framework for Description and Prediction." In *Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 1675–84. ACM.

Letham, Benjamin, Cynthia Rudin, Tyler H. McCormick, and David Madigan. 2015. "Interpretable Classifiers Using Rules and Bayesian Analysis: Building a Better Stroke Prediction Model." *The Annals of Applied Statistics* 9 (3): 1350–71.